



University of Frontier Technology, Bangladesh

Faculty of Software and Machine Intelligence Engineering

Department of Software Engineering

Software Requirements Specification

connectS

A Mobile-Based Academic Ecosystem for Institutional Academic Coordination

Version 1.0

Prepared by	
Names	Ruksana Akter Rojony (ID: 2303010)
	Shafinur Rahman (ID: 2303023)
	Emon Khondoker (ID: 2303032)
Department	Software Engineering, UFTB
Approved by	
Name	Md. Moshiur Rahman
Designation	Lecturer, Department of Software Engineering, UFTB

Submission Date: June 23, 2026

Transmittal Letter

June 23, 2026

The Course Instructor
Software Requirement Analysis (SE 206)
Department of Software Engineering
University of Frontier Technology, Bangladesh

Subject: Submission of the Report on "connectS — A Mobile-Based Academic Management System"

Dear Sir,

With all due respect, we would like to submit our report for the course Software Requirement Analysis (SE 206). This report presents the complete Software Requirements Specification (SRS) for our project **connectS**, a Flutter-based mobile application designed for academic institutions and departments.

The report covers the project background and description, requirements elicitation, Quality Function Deployment (QFD) analysis, use case diagrams, data object modeling, class-based modeling, architectural design, quality attributes, and a preliminary test plan.

We sincerely hope that you will find this report satisfactory and accept it as our course submission.

With best regards,

Ruksana Akter Rojony
ID: 2303010

Shafinur Rahman
ID: 2303023

Emon Khondoker
ID: 2303032

Department of Software Engineering, UFTB

Document Authentication

This project document has been prepared and approved by the following persons.

Prepared by

Approved by

Ruksana Akter Rojony

ID: 2303010

Department of Software Engineering,
UFTB

Md. Moshiur Rahman

Lecturer

Department of Software Engineering
University of Frontier Technology,
Bangladesh

Shafinur Rahman

ID: 2303023

Department of Software Engineering,
UFTB

Emon Khondoker

ID: 2303032

Department of Software Engineering,
UFTB

Acknowledgement

First of all, thanks to the Almighty Allah. By His grace, we have been able to complete this technical report on **connectS**, a mobile-based academic management system for academic institutions.

We are deeply grateful to our course instructor, **Md. Moshir Rahman**, Lecturer, Department of Software Engineering, University of Frontier Technology, Bangladesh, for his continuous guidance, valuable feedback, and encouragement throughout this project.

We also thank all the stakeholders who took part in our elicitation activities — the students, teachers, administrators, and staff — whose honest feedback helped us identify real problems and build a meaningful solution.

Finally, we thank our fellow students and friends who supported us with suggestions and constructive criticism during the preparation of this report.

Abstract

Academic institutions and departments around the world face significant challenges in managing communication, resources, and administrative workflows due to the lack of a unified digital platform. Students, teachers, administrators, and staff often rely on fragmented third-party tools such as WhatsApp groups, Facebook pages, and email chains, which lead to missed notices, expired resource links, delayed information dissemination, and inefficient administrative processes.

connectS is a Flutter-based mobile application designed to address these challenges by centralizing all departmental and institutional activities in one secure, role-based platform. The application supports multiple user roles including Student, Teacher, Class Representative (CR), Admin, and Office Staff. Key features include push notifications for assignment deadlines, a centralized notice and news feed, marks management, lecture slide and book repositories, a digital application and approval system, consultation scheduling, class routine management, a course catalogue, student profile integration, and dark/light mode support.

This report presents the complete Software Requirements Specification (SRS) for connectS. It includes the project background, requirements elicitation, Quality Function Deployment (QFD) analysis, use case diagrams, data object modeling with an entity relationship description, class-based modeling, architectural design, quality attributes, and a preliminary test plan.

Revision History

Version	Date	Author	Description of Change
1.0	June 23, 2026	Shafinur Rahman, Ruksana Akter Rojony, Emon Khon- doker	Initial version prepared for sub- mission.

Contents

Revision History	5
1 Introduction	10
1.1 Purpose	10
1.2 Document Conventions	10
1.3 Intended Audience	10
1.4 Project Scope	11
1.5 References	12
2 Project Introduction and Problem Identification	13
2.1 Project Title and Domain	13
2.2 Background of the System	13
2.3 Purpose and Objective	14
2.4 Stakeholder Analysis Matrix	14
2.5 Scope of the System	15
2.6 Gap in the Current System	15
2.7 Related Work Comparison	16
2.8 Real-Life Problem Scenarios	17
2.9 Root Causes of the Problem	18
2.10 Problem Statement	18
2.11 Justification for the Proposed Solution	19
3 Project Description	20
3.1 Quality Function Deployment (QFD)	20
3.1.1 Normal Requirements	20
3.1.2 Expected Requirements	20
3.1.3 Exciting Requirements	21
3.2 Motivation	21
4 Requirements Elicitation	23
4.1 Elicitation Methodology	23
4.1.1 Techniques Used	23
4.2 Key Findings from Elicitation	24
4.3 Raw Interview Summary	25
4.3.1 Student — Key Points from Interview	25
4.3.2 Class Representative (CR) — Key Points from Interview	25
4.3.3 Administrative Staff — Key Points from Interview	25
4.3.4 Teacher 1 — Key Points from Interview	25

4.3.5	Teacher 2 — Key Points from Interview	26
4.3.6	Teacher 3 — Key Points from Interview	26
4.4	Conflicting and Ambiguous Requirements	27
4.5	Prioritized Requirements	28
5	Product Use Case List	29
6	PUC-01: User Login & Authentication	31
7	PUC-02: Notice & News Feed	33
8	PUC-03: Assignment Deadline Management	35
9	PUC-04: Marks Management	37
10	PUC-05: Lecture Slide Upload & Access	39
11	PUC-06: Books PDF Corner	41
12	PUC-07: Class Routine Management	43
13	PUC-08: Teacher Schedule & Consultation Hours	45
14	PUC-09: Digital Application & Approval System	47
15	PUC-10: Student Online Handles Integration	49
16	PUC-11: Course Catalogue	51
17	PUC-12: Dark Mode / Light Mode	53
18	Development Sprint Plan Summary	55
19	Quality Attributes	57
19.1	Usability	57
19.2	Performance	57
19.3	Security	58
19.4	Safety	59
19.5	Other Quality Attributes	59
19.5.1	Availability	59
19.5.2	Reliability	60
19.5.3	Maintainability	60
19.5.4	Scalability	60
19.5.5	Portability	60

19.5.6 Interoperability	60
20 Internationalization and Localization Requirements	61
21 Other Requirements	62
21.1 Legal and Regulatory Compliance	62
21.2 Installation and Configuration	62
21.3 Logging and Monitoring	62
21.4 Transition Requirements	63
22 Specific Requirements	64
22.1 Functional Requirements	64
22.2 Non-Functional Requirements	65
23 Scenario Based Modeling	67
23.1 Use Case Diagrams	67
23.1.1 Level 0 — System Overview	67
23.1.2 Level 1 — User Login and Authentication	67
23.1.3 Level 2 — Assignment Deadline Management	68
23.1.4 Level 3 — Marks Management	68
23.1.5 Level 4 — Digital Application and Approval	69
24 Data and Class Based Modeling	70
24.1 Data Based Modeling	70
24.1.1 Identify Data Objects	70
24.1.2 Final Data Objects	70
24.2 Class Based Modeling	70
24.2.1 Final Classes	71
25 Architectural Design	73
25.1 Representing the System in Context	73
25.2 Mapping Requirements to Software Architecture	73
26 Testing Plan	75
26.1 High-Level Description of Testing Goals	75
26.2 Sample Test Cases	75
Conclusion	77
References	78
Appendix A: Glossary	79

Appendix B: Analysis Models	81
26.3 System Architecture in Context	81
26.4 Component Architecture	81
26.5 Entity-Relationship Diagram	81
26.6 Use Case Diagram — System Overview	82

1 Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for **connectS**, a Flutter-based mobile application developed for academic institutions and departments. It serves as both a short-form SRS and a design modeling report for the Software Requirement Analysis course (SE 206).

The document describes all functional and non-functional requirements of the system, organized with supporting diagrams and models. It is intended to give every reader a clear and complete understanding of what the system does, who uses it, and how it is structured.

1.2 Document Conventions

The following conventions are used throughout this document:

- **Product Use Cases (PUCs):** Numbered as PUC-XX, these describe specific user interactions with the system.
- **Functional Requirements (FRs):** Numbered as FR-XX, these define specific system behaviors.
- **Non-Functional Requirements (NFRs):** Numbered as NFR-XX, these represent quality attributes and constraints.
- **Quality Attributes (QAs):** Numbered as QA-XX, these define specific quality requirements.
- **User interface elements:** Shown in **bold** for buttons, menus, and screen elements.
- **Important notes:** Displayed in tcolorbox environments for emphasis.

1.3 Intended Audience

This SRS is intended for the following audiences:

- **Customers / Stakeholders:** Students, teachers, administrators, and staff of academic institutions will use this document to verify that the system meets their real needs.
- **Faculty Supervisor:** The course instructor will use this document to evaluate whether the team has correctly identified, analyzed, and modeled the system requirements.

- **Designers:** The design team will use the high-level architecture and data models to design the detailed system structure.
- **Developers:** Developers will refer to this document during implementation to ensure the system matches the stated requirements.
- **Testers:** The testing team will use this document to understand the expected behavior of each feature and design appropriate test cases.

1.4 Project Scope

Included in Scope:

- Secure institutional email-based registration and authentication system.
- Role-based access control for Admin, Teacher, Class Representative (CR), Student, and Office Staff users.
- Notice and news feed system for academic announcements and departmental updates.
- Assignment deadline notification and reminder system.
- Materials corner for sharing lecture slides, books, PDFs, and previous academic projects.
- Marks management system for academic performance tracking.
- Teacher schedule and consultation hour publishing system.
- Student online handle integration including GitHub, Codeforces, and other professional profiles.
- Class routine management system.
- Digital course catalogue containing academic course information.
- Digital application and approval system for student requests.
- Dark mode and light mode support.
- Mini quiz feature for in-class assessment.

Excluded from Scope:

- Integration with institutional financial or payment systems.
- Grading or automatic assessment systems.
- Online classroom systems or live video streaming.
- Features unrelated to academic communication and resource management.
- Integration with external institutional management systems in v1.0.
- Full offline mode support.

1.5 References

- [1] Pressman, R. S., & Maxim, B. R. (2015). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
- [2] IEEE. (1998). *IEEE Recommended Practice for Software Requirements Specifications* (IEEE Std 830-1998). IEEE Computer Society.
- [3] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education Limited.
- [4] Flutter Development Team. (2026). *Flutter Documentation*. Retrieved from <https://flutter.dev/docs>
- [5] Firebase. (2026). *Firebase Cloud Messaging Documentation*. Retrieved from <https://firebase.google.com/docs/cloud-messaging>
- [6] Wiegers, K., & Beatty, J. (2013). *Software Requirements* (3rd ed.). Microsoft Press.

2 Project Introduction and Problem Identification

2.1 Project Title and Domain

Project Title: "**connectS**: A Mobile-Based Academic Ecosystem for Institutional Academic Coordination"

Domain: EdTech and Academic Collaboration Systems. This is an institution-focused digital platform developed to support academic communication, collaborative learning, resource sharing, student networking, and community engagement within academic institutions and departments.

2.2 Background of the System

The majority of academic departments and educational institutions do not have a single online platform for students and teachers to easily communicate, collaborate and share academic resources. Existing systems, such as Moodle or university management portals, are mainly designed to support formal academic tasks, but they usually do not provide a friendly and interactive environment for teamwork, communication and student engagement.

Students and teachers usually use different platforms, such as Facebook groups, WhatsApp chats, Google Drive links and face-to-face interactions to share updates and learning materials. Information is spread around many locations so students may miss assignment deadlines, announcements, schedules or important academic updates.

The learning resources such as lecture slides, notes, books, previous study materials and coding practice resources are also difficult to organise and access properly. As student numbers and academic activities grow, communication and collaboration become more difficult.

To tackle these problems, we propose a system called **connectS**. It is a collaborative academic platform where students, teachers and the academic community can communicate, share resources, manage academic activities and stay connected in one centralised place.

2.3 Purpose and Objective

The main objective of **connectS** is to provide a centralised and collaborative academic platform for students and teachers to communicate easily, share academic contents and keep updated with important educational activities in a secure environment.

1. Develop a centralised system for notices, news feeds, assignment deadline notifications, class routines, course catalogues, and teacher schedules so that students and teachers can access important academic information easily and efficiently.
2. Create a digital platform where students can access books, PDFs, academic materials, and view marks in an organized and user-friendly manner.
3. Enable teachers to share schedules, academic materials, departmental events, and important faculty announcements efficiently.

2.4 Stakeholder Analysis Matrix

Stakeholder	Power Level	Interest Level	Management Strategy
Student	Low	High	Keep informed through notices, assignment notifications, academic resources, and collaborative features.
Teacher	High	High	Manage closely and involve in academic operations, resource sharing, and performance management.
Class Representative	Low	High	Keep informed and engaged for effective communication and class coordination activities.
Admin	High	High	Manage closely as the primary authority responsible for system maintenance, verification, and platform operations.
Guest	Low	Low	Monitor with minimal effort and provide limited access to public information only.

2.5 Scope of the System

Included in Scope:

- Secure institutional email-based registration and authentication system.
- Role-based access control for Admin, Teacher, Class Representative (CR), and Student users.
- Notice and news feed system for academic announcements and departmental updates.
- Assignment deadline notification and reminder system.
- Materials corner for sharing lecture slides, books, PDFs, and previous academic projects.
- Marks viewing system for academic performance tracking.
- Teacher schedule and consultation hour publishing system.
- Student online handle integration including GitHub, Codeforces, and other professional profiles.
- Class routine management system.
- Digital course catalogue containing academic course information.
- Centralized academic communication and collaboration platform for students and teachers.

Excluded from Scope:

- Integration with institutional financial or payment systems.
- Grading systems.
- Online classroom systems.
- Features unrelated to academic communication and resource management.

2.6 Gap in the Current System

Currently, many academic institutions are using several independent platforms, including institutional management systems, Moodle, Google Classroom, Facebook groups, Messenger chats, WhatsApp groups, shared drives, and oral communication modes to conduct academic activities and communicate. These systems support some educational operations, but they do not provide students and teachers with a unified and collaborative academic environment.

Institutional management systems focus mainly on formal administrative operations such as course registration, semester information and result publishing. Moodle and

Google Classroom are mainly used for assignment submission and sharing of course material, but they do not provide centralized academic communication, scheduling of consultations, academic networking and integrated student performance tracking. Informal platforms such as Facebook groups, WhatsApp, Messenger and Microsoft Teams are commonly used for communication and file sharing, but information is often scattered across different chats and groups, resulting in inefficient coordination and missed academic updates.

2.7 Related Work Comparison

Existing System	Purpose	Limitations
Institutional Management Systems	Used for formal institutional operations such as course registration, semester information, result publishing.	Does not offer collaborative communication features, resource sharing corners, assignment notifications, coding profile integration, or centralized academic interaction between students and teachers.
Moodle	Online learning management system, mainly used for course materials and assignment submission.	No integrated marks tracking. No centralised communication features. No coding profile integration. No repository of previous projects. No student community engagement features.
Google Classroom	Cloud-based classroom management platform for assignments and learning materials.	Limited academic customization and lacks features such as institutional email-based approval systems, centralized academic resource repositories, class routine management, and students' online coding handle integration.

Existing System	Purpose	Limitations
WhatsApp / Facebook Groups	Used informally for communication, messaging, and sharing of files.	Information becomes scattered across multiple groups and chats, causing missed deadlines, unorganized resources, lack of permanent academic archives, deadline notification and inefficient academic coordination.

2.8 Real-Life Problem Scenarios

Scenario 1 — The Silent Submission Disaster: A teacher posts an assignment deadline in the institutional learning management system. Several students miss the submission deadline despite completing the work. A centralized assignment notification system with proper reminders could prevent such situations.

Scenario 2 — Academic Archaeology Before Midterm: Before a midterm examination, a student searches for lecture slides, reference books, and previous project materials shared during earlier semesters. Since the resources are scattered across old chats, expired drive links, and personal collections, the student cannot gather the required materials efficiently. A centralized materials corner would make these resources permanently accessible within the blink of an eye, and organized.

Scenario 3 — The Mystery of the Blurry Marksheet: A teacher shares marks through a photo in a class group chat. Some students misread the marks while others cannot find the message later due to excessive chat activity. This creates confusion and repeated inquiries to the teacher. A secure centralized marks system would allow students to view their assessment scores more clearly and individually.

Scenario 4 — The Teacher Was Not There: A student knocks a teacher's door for academic consultation without knowing the teacher's absence. A centralized teacher schedule and consultation module would help students access teachers' schedules easily.

Scenario 5 — Searching for Mentors: Junior students interested in specific academic areas often struggle to find seniors working in similar areas because there is no common platform for showcasing academic profiles and interests. A student's online handles integration system could improve networking, collaboration, and peer learning opportunities within academic institutions.

2.9 Root Causes of the Problem

1. Absence of a dedicated, institution-owned digital platform for centralized information management.
2. Over-reliance on third-party social messaging platforms (WhatsApp, Facebook) that are not designed for academic administration.
3. Lack of role-based access control leading to unverified and unreliable information sharing.
4. No institutional policy mandating digital record-keeping for academic activities.
5. Insufficient technical infrastructure to support automated notifications or resource repositories.

2.10 Problem Statement

Academic institutions often use a few standalone tools for messaging, academic announcements, resource sharing, and collaboration between students and faculty. Consequently, key information such as assignment deadlines, notices, schedules, and academic materials are communicated through various channels, resulting in inefficiency and confusion among students and teachers. Existing academic systems are more orientated to some administrative or learning activities and do not provide a unified environment for centralised communication, academic resource management, consultation scheduling, performance tracking and student collaboration. This fragmented ecosystem leads to less academic engagement, communication breakdowns, and makes key academic resources difficult to organise and access consistently.

Thus, there is a need of a centralised and collaborative academic platform that can integrate communication, academic notifications, resource sharing, marks tracking, class routines, course catalogues, teacher consultation schedules and student academic networking within a single secure system.

2.11 Justification for the Proposed Solution

Problem	How connectS Solves It
Missed assignment deadlines and academic updates	Assignment deadline notifications and centralized news feed alerts help students stay informed about academic activities and important announcements.
Scattered notices and fragmented communication	A centralized Notice & News Feed System allows Admins, Teachers, and CRs to publish official announcements in one organized platform.
Difficulty accessing study materials	The Materials Corner provides organized access to lecture slides, books, PDFs, and previous academic projects in a centralized repository.
Lack of organized marks access	The Marks System allows students to securely and easily view their academic performance course-wise.
Consultation schedule confusion	The Teacher Schedule & Consultation Hours module enables teachers to publish and update consultation schedules for students.
Lack of centralized academic information	Class Routine and Course Catalogue modules provide organized access to academic schedules and course-related information.
Limited academic networking and collaboration	Student Online Handles Integration allows students to connect professional profiles to encourage collaboration and peer learning.
Poor preservation of academic resources and previous works	Centralized storage of books, slides, and previous projects ensures long-term accessibility and organized academic content management.

3 Project Description

3.1 Quality Function Deployment (QFD)

Quality Function Deployment (QFD) is a method that translates customer needs into obvious technical requirements. To gather requirements for connectS, the team conducted interviews with students, teachers, administrators, and staff, and distributed surveys to gather quantitative data.

3.1.1 Normal Requirements

Normal requirements are the basic features that users expect. If these are missing, users will be dissatisfied.

1. Institutional email-based login with role assignment (Student, Teacher, CR, Admin, Office Staff).
2. A centralized notice board where Admin posts official notices and CR posts batch notices.
3. Assignment deadline display with due date and course information.
4. Marks entry by teachers and marks viewing by students (own marks only).
5. Lecture slide upload by teachers, organized by course, stored permanently.
6. Books PDF library organized by course.
7. Class routine, kept updated by Admin each semester.
8. Course catalogue with course code, credits, semester, prerequisites, and teacher name.
9. Teacher schedule and consultation hour display on the teacher profile page.
10. Student academic/professional handling on student profile.

3.1.2 Expected Requirements

Expected requirements are features users assume will be present without saying so directly.

1. Secure login with institutional email domain verification and bcrypt password hashing.
2. Role-based access control — each user sees only the features allowed for their role.
3. Fast loading on slow networks (under 3 seconds on a 3G connection).

4. Simple and easy-to-use interface that requires no training for new users.
5. All uploaded content is stored permanently and never deleted automatically.

3.1.3 Exciting Requirements

Exciting requirements are features users did not explicitly ask for but will be very happy to find.

1. **Digital Application and Approval System:** Students submit applications through the app. Administrators forward them to the right authority. Authorized personnel give digital approval even if they are not physically present.
2. **Dark Mode and Light Mode:** All stakeholder groups independently requested this during interviews. Users can switch themes and their preference is saved permanently.
3. **Mini Quiz Feature:** Teachers can create short multiple-choice quizzes inside the app .
4. **Next-Day Consultation Availability:** Teachers can update their free time slots for the following day in the evening, so students can plan ahead without messaging the teacher.

3.2 Motivation

The motivation for building connectS comes directly from real problems confirmed during stakeholder interviews and surveys:

- Notices are spread across multiple communication channels. Students regularly miss important updates.
- Lecture slides and book links shared via informal channels expire within a short period and are not re-shared.
- Students do not learn their marks until the teacher distributes printed sheets, sometimes weeks after the test.
- Existing institutional systems delete uploaded content after a few months, leaving no permanent record.
- Student applications submitted through administrative channels get stuck when authorized personnel are unavailable.
- Teachers receive numerous unnecessary messages from students just asking when they are free for consultation.

- Class representative notices sent via personal broadcasts are unreliable and may not reach all students.
- Teachers and administrators do not want to use multiple apps. They want one combined platform.

4 Requirements Elicitation

4.1 Elicitation Methodology

The team employed multiple techniques to gather comprehensive requirements from all stakeholders.

4.1.1 Techniques Used

1. **Stakeholder Interviews:** The team talked directly with students, teachers, administrators, and staff. Each person was asked open questions about the problems they face and what they need from an institutional app.
2. **Voice Recording:** All interviews were recorded with the participant's permission. This made sure no important point was missed.
3. **Observation:** The team watched how notices are shared in existing communication channels and how students look for materials, in order to understand the real problems.
4. **Brainstorming Sessions:** The team discussed the collected feedback together and matched each problem to a feature in the app.
5. **Document Analysis:** The team looked at existing documents like class routines, course lists, and mark sheets to understand what data the app needs to handle.
6. **Student Survey:** A questionnaire was distributed to gather quantitative data on feature priorities.

4.2 Key Findings from Elicitation

#	Finding	Mapped Feature
1	Students miss notices regularly because messages get buried in busy communication channels.	Notice & News Feed System
2	Schedule changes do not always reach students on time.	Class Routine with Push Notifications
3	Lecture slides are uploaded in different places by different teachers.	Lecture Slide Upload
4	Students, CR, and teachers all asked for dark mode and light mode support.	Dark Mode / Light Mode
5	Student applications sent through administrative channels get stuck when authorized personnel are unavailable.	Digital Application & Approval System
6	Teachers want a consultation system where students can see next-day availability.	Teacher Schedule & Consultation Hours
7	Course content uploaded to institutional systems disappears after a few months.	Permanent Slides and Books Repository
8	Teachers do not want to use many different apps; they want one combined platform.	Unified connectS Platform
9	Teachers want to track student growth using marks, assignment grades, and assessments together.	Marks Management System
10	A mini quiz feature was requested to allow quick in-class student assessment.	Mini Quiz Feature

4.3 Raw Interview Summary

4.3.1 Student — Key Points from Interview

- "If a class schedule changes and the teacher updates the time, that notice should come directly through the app."
- "Having one common platform for everything would be great. Right now everything is scattered."
- "Different teachers upload slides in different places. If everything was in one mobile app, it would be much easier."
- "Dark mode and light mode — both options should be there."

4.3.2 Class Representative (CR) — Key Points from Interview

- "When a teacher changes their class time, the CR has to manually tell everyone. If the app sends that notice directly, the CR's workload goes down a lot."
- "If there is a class notice, I should be able to log in with my CR role and post it directly in the app for my batch."
- "Dark and light mode should both be there."

4.3.3 Administrative Staff — Key Points from Interview

- "Right now, students submit physical applications through the administrative office."
- "If authorized personnel are on leave, the application just sits there. Nobody processes it."
- "If applications could be uploaded in the app, authorized personnel could see them and give digital approval even if they are not in the office."
- "Dark and light mode should both be available."

4.3.4 Teacher 1 — Key Points from Interview

- "You need to study what already exists and then add 2 or 3 unique features that are not in other systems."
- "The app must have lecture upload, marks management, and assessment tracking so teachers can monitor student growth."
- "It must be secure. And the solution should be user-friendly, not developer-friendly."

4.3.5 Teacher 2 — Key Points from Interview

- "A consultation system would be great. The teacher updates the next day's free time the evening before."
- "Students should be able to find the teacher's contact information like email easily in the app."
- "Our institutional systems lose uploaded content after a few months. The app must keep a permanent backup."
- "A mini quiz feature in the app would be very useful."

4.3.6 Teacher 3 — Key Points from Interview

- "Using multiple apps is difficult. They should be combined into one platform."
- "Notification reminders are very useful."
- "A mobile app is good if the security is strong."
- "Teachers should be able to upload marks directly."
- "Both dark and light mode should be available. The app should be user-friendly and secure."

4.4 Conflicting and Ambiguous Requirements

#	Conflict / Ambiguity	Stakeholders Involved	Resolution
1	Some students wanted to see all classmates' marks to compare. Teachers said this breaks student privacy.	Students vs. Teachers	Resolved: Students see only their own marks. A leaderboard may be added in a future version.
2	Teachers asked for assignment file submission. The team said this is a large addition for v1.0.	Teachers vs. Project Team	Resolved: Only deadline tracking is included in v1.0. File submission is planned for v2.0.
3	Students expected the app to work offline. Teachers and Admin said real-time data cannot be offline.	Students vs. Teachers / Admin	Resolved: The app is online-only in v1.0 with a clear message when there is no connection.
4	CR wanted to post notices to all batches. Admin wanted posting limited to the CR's own batch only.	CR vs. Admin	Resolved: CR can only post to their own batch. Admin posts institution-wide notices.
5	Teachers requested integration with existing institutional management systems. This is a large integration.	Teachers vs. Project Scope	Resolved: connectS will work as a standalone app in v1.0. Integration is listed as a future enhancement.
6	The term "assessment marks" meant different things to different teachers.	Teachers (internal conflict)	Resolved: Assessment marks in the system covers all continuous assessment components as defined by each teacher per course.

4.5 Prioritized Requirements

Requirements are prioritized using the **MoSCoW** method.

Requirement	Priority	Justification
Institutional Email Authentication	Must Have	All features depend on verified identity.
Assignment Deadline Notifications	Must Have	Top-rated feature; directly solves the biggest student pain point.
Marks Management System	Must Have	Critical for students and reduces teacher manual work.
Notice & News Feed System	Must Have	Replaces the fragmented communication system.
Class Routine Management	Must Have	Needed every day by all students.
Lecture Slide Upload	Must Have	Raised by students, CR, and multiple teachers as a top need.
Digital Application & Approval System	Must Have	Solves a real administrative workflow problem.
Dark Mode / Light Mode	Must Have	Requested by all stakeholder groups.
Teacher Schedule & Consultation Hours	Should Have	Reduces unnecessary messages to teachers.
Books PDF Corner	Should Have	Solves the expired link problem for reference books.
Course Catalogue	Should Have	Useful reference; low implementation difficulty.
Mini Quiz Feature	Could Have	Useful but not urgent for v1.0.
Student Online Handles Integration	Could Have	Useful for academic profiling; low daily urgency.
Institutional System Integration	Won't Have (v1.0)	Too large for this version; planned as a future enhancement.
Full Offline Mode	Won't Have (v1.0)	Real-time data makes full offline mode impractical in v1.0.

5 Product Use Case List

PUC ID	Use Case Name	Primary Actor	Brief Description
PUC-01	User Login & Authentication	All users	User logs in with institutional email and is given role-based access.
PUC-02	Notice & News Feed	Admin, CR, All users	Admin/CR posts notices; all users read and get notified.
PUC-03	Assignment Deadline Management	Teacher, Student	Teacher posts assignment; students get notified and track deadlines.
PUC-04	Marks Management	Teacher, Student	Teacher enters and publishes marks; student views own marks.
PUC-05	Lecture Slide Upload & Access	Teacher, Student	Teacher uploads slides by course; students view and download.
PUC-06	Books PDF Corner	Teacher, Admin, Student	Teacher/Admin uploads books by course; students access them.
PUC-07	Class Routine Management	Admin, All users	Admin updates semester routine; all users view latest version.
PUC-08	Teacher Schedule & Consultation	Teacher, Student	Teacher posts free hours; student checks and visits for help.
PUC-09	Digital Application & Approval	Student, Office Staff, Teacher	Student submits application; office staff forwards; teacher approves.
PUC-10	Student Online Handles	Student, Teacher	Student adds professional handles; teacher views profiles.

PUC ID	Use Case Name	Primary Actor	Brief Description
PUC-11	Course Catalogue	Admin, All users	Admin maintains course list; all users browse course information.
PUC-12	Dark Mode / Light Mode	All users	User switches display theme; app saves the preference.

6 PUC-01: User Login & Authentication

Product Use Case — PUC-01

Use Case ID	PUC-01
Use Case Name	User Login & Authentication
Primary Actor	All users (Student, Teacher, CR, Admin, Office Staff)
Goal	A verified institutional member logs in and gets access to the right features for their role.
Precondition	The user has a valid institutional email address.
Main Flow	User opens the app → enters institutional email and password → system checks email domain → system confirms role → user sees their role-specific home screen.
Alt. Flow	If the email is not from the allowed domain, the system shows an error and blocks access.
Postcondition	User is logged in and can only access features allowed for their role.

Functional Requirements — PUC-01

- FR-01.1** The system must check that the user's email belongs to the allowed institutional domain before allowing login.
- FR-01.2** The system must assign one of five roles (Student, Teacher, CR, Admin, Office Staff) to each account.
- FR-01.3** The system must show a different home screen and menu based on the user's role.
- FR-01.4** The system must not allow users to access features outside their role.
- FR-01.5** The system must store passwords using a secure hashing method (e.g., bcrypt).
- FR-01.6** The system must keep the user logged in between sessions unless they manually log out.

Completion Plan — PUC-01

Step 1: Design: Create the login screen UI with email and password fields. Add email domain validation logic.

Step 2: Backend: Set up user registration and login API. Implement bcrypt password hashing. Store role with each user record in the database.

Step 3: Role Routing: Build role-based navigation so each role sees the correct screen after login.

Step 4: Testing: Test with valid institutional email, invalid email, wrong password, and each role type.

Step 5: Completion Target: Week 2 of development sprint.

7 PUC-02: Notice & News Feed

Product Use Case — PUC-02

Use Case ID	PUC-02
Use Case Name	Notice & News Feed
Primary Actor	Admin (official notices), CR (batch notices), All users (reading)
Goal	All institution members receive important notices in one place through the app.
Precondition	User is logged in. Admin or CR has a notice to post.
Main Flow	Admin/CR opens the notice section → writes a notice → posts it → system sends push notification to all relevant users → users read the notice in the feed.
Alt. Flow	CR can only post to their own batch. Admin can post to the whole institution.
Postcondition	Notice is visible in the feed and all targeted users have been notified.

Functional Requirements — PUC-02

- FR-02.1** The system must allow Admin to post official notices visible to all institution users.
- FR-02.2** The system must allow CR to post notices visible only to their assigned batch.
- FR-02.3** The system must send a push notification to all targeted users when a new notice is posted.
- FR-02.4** The system must display notices in reverse chronological order (newest first).
- FR-02.5** The system must allow users to tap a notification and go directly to the notice detail screen.
- FR-02.6** The system must store all notices permanently so old notices can be viewed later.

Completion Plan — PUC-02

Step 1: Design: Create notice feed UI screen and notice compose screen for Admin and CR.

Step 2: Backend: Build notice creation API. Store notices with role/batch targeting in the database.

Step 3: Notifications: Integrate Firebase Cloud Messaging (FCM) to send push notifications on new notice creation.

Step 4: Access Control: Add server-side check so CR notices are only shown to the correct batch.

Step 5: Testing: Test posting as Admin vs. CR; verify push notification delivery; test batch filtering.

Step 6: Completion Target: Week 3 of development sprint.

8 PUC-03: Assignment Deadline Management

Product Use Case — PUC-03

Use Case ID	PUC-03
Use Case Name	Assignment Deadline Management
Primary Actor	Teacher (creates), Student (receives)
Goal	Students never miss assignment deadlines because the app automatically notifies them.
Precondition	Teacher is logged in and has an assignment to post.
Main Flow	Teacher creates an assignment with title, description, course, and deadline → system notifies all students in that course → students see the assignment in their list → system sends a reminder 24 hours before the deadline.
Alt. Flow	If the deadline passes, the assignment is marked as expired in the student's view.
Postcondition	Assignment is saved, students are notified, and reminders are scheduled.

Functional Requirements — PUC-03

- FR-03.1** The system must allow teachers to create assignments with a title, description, course name, and deadline date and time.
- FR-03.2** The system must send a push notification to all students in the course within 1 minute after the assignment is posted.
- FR-03.3** The system must send an automatic reminder notification to students 24 hours before the deadline.
- FR-03.4** The system must display all active assignments sorted by deadline (closest first) in the student's view.
- FR-03.5** The system must mark assignments as expired after the deadline passes.
- FR-03.6** Only the teacher who created the assignment or an Admin can change the deadline.

Completion Plan — PUC-03

Step 1: Design: Build the assignment creation form for teachers and the assignment list screen for students.

Step 2: Backend: Create assignment API (create, read, update). Store deadline as a timestamp.

Step 3: Scheduler: Set up a background job to send reminder notifications 24 hours before each deadline using FCM.

Step 4: Expiry Logic: Add logic to automatically mark assignments as expired when the deadline passes.

Step 5: Testing: Test creation, notification timing, expiry marking, and permission check for deadline edits.

Step 6: Completion Target: Week 4 of development sprint.

9 PUC-04: Marks Management

Product Use Case — PUC-04

Use Case ID	PUC-04
Use Case Name	Marks Management
Primary Actor	Teacher (enters marks), Student (views own marks)
Goal	Students can check their marks anytime without waiting for a printed sheet.
Precondition	Teacher has finished marking and is ready to publish results.
Main Flow	Teacher opens the marks entry screen → selects course and assessment type → enters marks for each student → publishes → students get a notification → each student sees only their own mark.
Alt. Flow	Teacher can save marks as a draft and publish later.
Postcondition	Marks are saved, published, and visible to the correct student only.

Functional Requirements — PUC-04

- FR-04.1** The system must allow teachers to enter marks per student per course.
- FR-04.2** The system must allow teachers to save marks as a draft before publishing.
- FR-04.3** The system must show marks to students only after the teacher explicitly publishes them.
- FR-04.4** Each student must see only their own marks. No student can view another student's marks.
- FR-04.5** The system must send a push notification to students when their marks are published.
- FR-04.6** The system must display marks clearly with the course name, assessment type, and score.

Completion Plan — PUC-04

Step 1: Design: Build marks entry screen for teachers and marks view screen for students.

Step 2: Backend: Create marks API with draft and published states. Store marks linked to student ID, course, and assessment type.

Step 3: Privacy Logic: Add server-side filter so each API call returns only the requesting student's own marks.

Step 4: Notification: Trigger FCM push notification to enrolled students when marks are published.

Step 5: Testing: Test draft save, publish trigger, privacy isolation, and notification delivery.

Step 6: Completion Target: Week 5 of development sprint.

10 PUC-05: Lecture Slide Upload & Access

Product Use Case — PUC-05

Use Case ID	PUC-05
Use Case Name	Lecture Slide Upload & Access
Primary Actor	Teacher (uploads), Student (views and downloads)
Goal	All lecture slides are stored permanently in one place organized by course so students can always find them.
Precondition	Teacher is logged in and has a slide file to upload.
Main Flow	Teacher selects a course → uploads slide file with a title → slide appears in the course's slide list → students open the course and find all uploaded slides.
Alt. Flow	If the file is too large, the system shows an error and asks for a smaller file.
Postcondition	Slide is saved permanently and visible to all students enrolled in the course.

Functional Requirements — PUC-05

- FR-05.1** The system must allow teachers to upload slide files (PDF or PPTX) linked to a specific course.
- FR-05.2** The system must store all uploaded slides permanently and never delete them automatically.
- FR-05.3** The system must display slides in a list under the correct course, sorted by upload date.
- FR-05.4** Students must be able to view slides inside the app or download them to their device.
- FR-05.5** Only the teacher who uploaded a slide or an Admin can delete or replace it.
- FR-05.6** The system must show the upload date and slide title in the list view.

Completion Plan — PUC-05

Step 1: Design: Build slide upload screen for teachers and slide list/viewer screen for students.

Step 2: Storage: Set up cloud file storage for permanent hosting.

Step 3: Backend: Create slide upload and retrieval API. Store file URL, course ID, title, and upload date in the database.

Step 4: Viewer: Integrate an in-app PDF viewer so students can read slides without leaving the app.

Step 5: Testing: Test upload, permanent storage, course filtering, in-app viewing, and download.

Step 6: Completion Target: Week 5 of development sprint.

11 PUC-06: Books PDF Corner

Product Use Case — PUC-06

Use Case ID	PUC-06
Use Case Name	Books PDF Corner
Primary Actor	Teacher, Admin (upload), Student (view)
Goal	Students can easily find the correct course reference book without using old links.
Precondition	Teacher or Admin has a book PDF to upload.
Main Flow	Teacher/Admin selects a course → uploads a book PDF with title and author name → the book is added to the course book list → students open the course and read the book.
Alt. Flow	Admin can remove or replace an outdated book.
Postcondition	Book is stored permanently and accessible to all logged-in students.

Functional Requirements — PUC-06

- FR-06.1** The system must allow teachers and Admins to upload book PDFs linked to a specific course.
- FR-06.2** The system must display books in a list organized by course with book title and author name.
- FR-06.3** Students must be able to open books inside the app using an in-app PDF viewer.
- FR-06.4** All book files must be stored permanently and never expire.
- FR-06.5** Only teachers or Admins can upload, replace, or remove books.
- FR-06.6** The system must support lazy loading so large PDFs do not slow down the app on opening.

Completion Plan — PUC-06

Step 1: Design: Build book upload screen and book library screen organized by course.

Step 2: Storage: Use same cloud storage as slides. Store book metadata (title, author, course, URL) in database.

Step 3: Backend: Create book upload and retrieval API with course-based filtering.

Step 4: Viewer: Implement lazy-loading PDF viewer to handle large book files smoothly.

Step 5: Testing: Test upload, course filtering, lazy loading performance, and access control.

Step 6: Completion Target: Week 6 of development sprint.

12 PUC-07: Class Routine Management

Product Use Case — PUC-07

Use Case ID	PUC-07
Use Case Name	Class Routine Management
Primary Actor	Admin (updates), All users (view)
Goal	All students and teachers can see the latest class schedule in the app at any time, including when a change is made.
Precondition	Admin is logged in.
Main Flow	Admin opens the routine section → uploads or updates the semester routine → all users can see the latest version immediately.
Alt. Flow	If a teacher changes a class time, Admin updates the routine and all users get a push notification.
Postcondition	The latest routine is visible to all users and old versions are saved as history.

Functional Requirements — PUC-07

- FR-07.1** The system must allow Admin to upload and update the class routine.
- FR-07.2** The system must display the latest routine to all logged-in users immediately after an update.
- FR-07.3** The system must send a push notification to all users when the routine is updated.
- FR-07.4** The system must keep a history of old routines so past versions can be viewed if needed.
- FR-07.5** The routine must be clearly organized by day and time slot.
- FR-07.6** Only Admin can update or replace the class routine.

Completion Plan — PUC-07

Step 1: Design: Build the routine view screen showing a weekly timetable grid.
Build an Admin upload/edit screen.

Step 2: Backend: Create routine API (upload, update, fetch latest, fetch history). Store each version with a timestamp.

Step 3: Notification: Trigger FCM push notification to all users when routine is updated.

Step 4: History: Add a routine history screen so users can view previous semester routines.

Step 5: Testing: Test update flow, push notification, history retrieval, and non-admin access restriction.

Step 6: Completion Target: Week 4 of development sprint.

13 PUC-08: Teacher Schedule & Consultation Hours

Product Use Case — PUC-08

Use Case ID	PUC-08
Use Case Name	Teacher Schedule & Consultation Hours
Primary Actor	Teacher (posts schedule), Student (views)
Goal	Students can check when a teacher is free without sending a message or making a call.
Precondition	Teacher is logged in.
Main Flow	Teacher opens their profile → updates their weekly schedule and free consultation slots → students open the teacher's profile → students see the current schedule and free hours.
Alt. Flow	Teacher can update next-day availability the evening before for short-notice consultation.
Postcondition	Teacher's schedule is live and visible to all logged-in students.

Functional Requirements — PUC-08

- FR-08.1** The system must allow teachers to post and update their weekly teaching schedule and free consultation hours.
- FR-08.2** The system must display the teacher's schedule on their profile page, visible to all logged-in students.
- FR-08.3** The system must allow teachers to update next-day availability separately for short-notice consultation slots.
- FR-08.4** The system must display the teacher's contact email on their profile page.
- FR-08.5** Schedule updates must be reflected immediately without requiring the student to refresh manually.
- FR-08.6** Only the teacher themselves can edit their own schedule.

Completion Plan — PUC-08

Step 1: Design: Build teacher profile screen showing name, email, schedule, and consultation hours. Build schedule edit screen for teachers.

Step 2: Backend: Create teacher schedule API (update, fetch). Store weekly slots and next-day availability separately.

Step 3: Real-time Update: Use polling or WebSocket so students see schedule changes without manual refresh.

Step 4: Testing: Test schedule update, next-day availability update, contact email display, and edit permission check.

Step 5: Completion Target: Week 6 of development sprint.

14 PUC-09: Digital Application & Approval System

Product Use Case — PUC-09

Use Case ID	PUC-09
Use Case Name	Digital Application & Approval System
Primary Actor	Student (submits), Office Staff (forwards), Teacher/Admin (approves)
Goal	Student applications are processed quickly and do not get stuck when authorized personnel are not physically present.
Precondition	Student is logged in and has an application to submit.
Main Flow	Student fills the application form in the app → submits it → office staff sees it on their dashboard → sends it to the correct authority → authority approves it digitally → student receives a notification with the result.
Alt. Flow	If the assigned authority is unavailable, office staff can reassign to another appropriate person.
Postcondition	Application is approved or rejected; student is notified; a record is saved.

Functional Requirements — PUC-09

- FR-09.1** The system must allow students to fill in and submit application forms through the app.
- FR-09.2** The system must show all submitted applications in the office staff dashboard with student name, date, and type.
- FR-09.3** The system must allow office staff to assign an application to a specific authority.
- FR-09.4** The system must notify the assigned authority when an application is sent to them for approval.
- FR-09.5** The system must allow the authority to approve or reject the application with a comment.
- FR-09.6** The system must notify the student when their application has been approved or rejected.
- FR-09.7** The system must save a record of all applications and their final status permanently.

Completion Plan — PUC-09

Step 1: Design: Build application form screen for students, application dashboard for office staff, and approval screen for authorities.

Step 2: Backend: Create application API with status states: Submitted → Forwarded → Approved/Rejected. Store all records permanently.

Step 3: Notification: Send FCM notifications at each status change (submission, forwarding, approval/rejection).

Step 4: Reassignment: Add logic for office staff to reassign if the original authority is marked as unavailable.

Step 5: Testing: Test full flow from submission to approval; test reassignment case; verify notification at each step.

Step 6: Completion Target: Week 7 of development sprint.

15 PUC-10: Student Online Handles Integration

Product Use Case — PUC-10

Use Case ID	PUC-10
Use Case Name	Student Online Handles Integration
Primary Actor	Student (adds handles), Teacher (views)
Goal	Teachers can see each student's professional and academic activity in one place for assessment and mentorship.
Precondition	Student is logged in and has platform handles to add.
Main Flow	Student opens profile settings → adds professional handles → teacher opens the student's profile → teacher sees the handles as clickable links.
Alt. Flow	Student can leave any handle blank if they do not use that platform.
Postcondition	Handles are saved and visible to teachers and Admins as clickable links.

Functional Requirements — PUC-10

- FR-10.1** The system must allow students to add, update, and remove their professional handles from profile settings.
- FR-10.2** The system must display these handles on the student's profile as clickable links that open the correct profile page.
- FR-10.3** Handles must be visible to all teachers and Admins but not to other students.
- FR-10.4** The system must not make handles mandatory; any field can be left blank.
- FR-10.5** Handle links must open in the device's default browser when tapped.

Completion Plan — PUC-10

Step 1: Design: Add professional handle input fields to the student profile settings screen.

Step 2: Backend: Store handles in the user profile record. Build API to update and fetch handles.

Step 3: Access Control: Restrict handle visibility to Teacher and Admin roles only on the server side.

Step 4: Link Handling: Build dynamic URL generation and open with external browser on tap.

Step 5: Testing: Test adding, updating, and removing handles; test link opening; test visibility restriction.

Step 6: Completion Target: Week 7 of development sprint.

16 PUC-11: Course Catalogue

Product Use Case — PUC-11

Use Case ID	PUC-11
Use Case Name	Course Catalogue
Primary Actor	Admin (manages), All users (browse)
Goal	All users can browse the full list of courses and find course details easily.
Precondition	Admin has added courses to the system.
Main Flow	User opens the course catalogue → browses courses by semester or searches by name → taps a course to see details (code, credits, prerequisites, teacher).
Alt. Flow	Admin can add, update, or archive courses each semester.
Postcondition	User has found the course information they needed.

Functional Requirements — PUC-11

- FR-11.1** The system must display a list of all courses showing course code, title, credits, and semester.
- FR-11.2** Users must be able to tap a course to see its full details including prerequisites and assigned teacher.
- FR-11.3** Users must be able to search for a course by name or code.
- FR-11.4** Users must be able to filter courses by semester number.
- FR-11.5** Only Admin can add, update, or remove courses from the catalogue.

Completion Plan — PUC-11

Step 1: Design: Build the course catalogue list screen with search and semester filter. Build course detail screen.

Step 2: Backend: Create course API (add, update, fetch, archive). Store all course fields in the database.

Step 3: Search/Filter: Implement client-side search by name/code and filter by semester.

Step 4: Admin Panel: Add course management section to the Admin panel for adding and updating courses.

Step 5: Testing: Test search, filter, detail view, Admin CRUD operations, and access restriction.

Step 6: Completion Target: Week 6 of development sprint.

17 PUC-12: Dark Mode / Light Mode

Product Use Case — PUC-12

Use Case ID	PUC-12
Use Case Name	Dark Mode / Light Mode
Primary Actor	All users
Goal	Users can switch the app display between dark and light themes to suit their comfort and lighting conditions.
Precondition	User is logged in and in the app settings.
Main Flow	User opens settings → taps the theme toggle → app switches to the chosen theme immediately → the chosen theme is saved and used next time the app opens.
Alt. Flow	If the user has not set a preference, the app follows the phone's system-level dark/light setting by default.
Postcondition	Theme preference is saved and applied consistently across all screens.

Functional Requirements — PUC-12

- FR-12.1** The system must provide a toggle switch in the settings screen to switch between dark and light mode.
- FR-12.2** The theme change must apply to all screens immediately when the toggle is switched.
- FR-12.3** The system must save the user's theme preference locally so it is applied every time the app opens.
- FR-12.4** If no preference is set, the app must follow the device's system-level theme setting by default.
- FR-12.5** All screens, text, icons, and backgrounds must be properly styled for both dark and light themes.

Completion Plan — PUC-12

- Step 1: Design:** Define a complete dark theme and light theme color palette in the Flutter theme system.
- Step 2: Settings Screen:** Add a theme toggle switch to the app settings screen.
- Step 3: State Management:** Use a global state manager to apply the theme change across all screens immediately.
- Step 4: Persistence:** Save the theme preference using local storage and load it on app startup.
- Step 5: System Default:** Use Flutter's `MediaQuery.platformBrightness` to detect the device theme when no preference is saved.
- Step 6: Testing:** Test theme toggle on all major screens; test persistence after app restart; test system default behavior.
- Step 7: Completion Target:** Week 3 of development sprint.

18 Development Sprint Plan Summary

PUC ID	Use Case Name	Target Week	Key Milestone
PUC-01	User Login & Authentication	Week 1	Login and role routing working
PUC-12	Dark Mode / Light Mode	Week 1	Theme applied across all screens
PUC-02	Notice & News Feed	Week 1	Push notifications working
PUC-07	Class Routine Management	Week 2	Routine update and notification working
PUC-03	Assignment Deadline Management	Week 2	Auto-reminder notifications working
PUC-04	Marks Management	Week 2	Draft/publish flow and privacy working
PUC-05	Lecture Slide Upload & Access	Week 2	Permanent storage and in-app viewer working
PUC-06	Books PDF Corner	Week 2	Lazy loading PDF viewer working
PUC-08	Teacher Schedule & Consultation	Week 3	Real-time schedule update working
PUC-11	Course Catalogue	Week 3	Search and filter working
PUC-09	Digital Application & Approval	Week 3	Full submission-to-approval flow working
PUC-10	Student Online Handles Integration	Week 3	Links opening correctly, access control verified

Overall Completion Plan Summary

- **Total PUCs:** 12
- **Total Functional Requirements:** 64
- **Development Duration:** 7 weeks
- **Technology:** Flutter (frontend), REST API backend, Firebase Cloud Messaging (notifications), Cloud Storage (files), SharedPreferences (local theme storage)
- **Testing Approach:** Each PUC is tested individually after its target week. Integration testing is done in Week 8 before final submission.
- **Priority Order:** Authentication → Notices → Routine → Assignments → Marks → Resources → Applications → Profiles

19 Quality Attributes

19.1 Usability

The following usability requirements ensure that the application is easy to learn, efficient to use, and accessible to all users:

- US-1.** The application shall be intuitive and require no formal training for first-time users.
- US-2.** Key features shall be reachable within two taps from the home screen.
- US-3.** All error messages shall be clear, actionable, and provide guidance for resolution.
- US-4.** The interface shall support both dark and light themes for user comfort in different lighting conditions.
- US-5.** Accessibility features shall support users with visual impairments (e.g., sufficient contrast, scalable fonts).
- US-6.** The application shall provide consistent navigation patterns across all screens.
- US-7.** Touch targets shall be at least 44x44 points for easy interaction on mobile devices.
- US-8.** The application shall support screen reader compatibility for visually impaired users.
- US-9.** Loading indicators shall be displayed for operations taking more than 2 seconds.
- US-10.** Confirmation dialogs shall be displayed for destructive actions (e.g., delete, publish).

19.2 Performance

Performance requirements define the speed, responsiveness, and resource utilization of the system:

- PE-1.** The application's main screen shall load within 3 seconds on a stable 3G network connection.
- PE-2.** The API shall respond to requests within 2 seconds for 95% of all transactions.
- PE-3.** Push notifications shall be delivered to users within 60 seconds of the triggering event.

- PE-4.** File uploads shall support files up to 50MB in size.
- PE-5.** The application shall support at least 500 concurrent users without performance degradation.
- PE-6.** Lazy loading shall be implemented for large PDF files (over 10MB).
- PE-7.** The application shall consume no more than 100MB of device storage for cached data.
- PE-8.** Screen transitions shall complete within 300 milliseconds.
- PE-9.** The application shall maintain 60 frames per second (fps) for smooth animations.
- PE-10.** Database queries shall execute within 500 milliseconds for 95% of operations.

19.3 Security

Security requirements ensure the confidentiality, integrity, and availability of system data:

- SE-1.** All passwords shall be stored using bcrypt hashing with a work factor of 10 or higher.
- SE-2.** All data transmission between the application and backend shall use HTTPS/TLS 1.2 or higher encryption.
- SE-3.** Role-based access control shall be enforced at both the application and API levels.
- SE-4.** JSON Web Tokens (JWT) shall be used for API authentication with a 30-minute expiration.
- SE-5.** Session timeout shall occur after 30 minutes of inactivity.
- SE-6.** User data shall be protected from unauthorized access through server-side validation.
- SE-7.** All API endpoints shall validate user permissions before processing requests.
- SE-8.** Failed login attempts shall be limited to 5 attempts before temporary account lockout.
- SE-9.** Password recovery shall require verification through institutional email.

- SE-10.** All security events shall be logged for audit purposes (e.g., login attempts, permission changes).

19.4 Safety

Safety requirements address risk prevention and system reliability:

- SA-1.** The system shall prevent accidental data loss through confirmation dialogs for destructive actions.
- SA-2.** Drafts shall be automatically saved every 30 seconds to prevent loss of entered data.
- SA-3.** The application shall handle network errors gracefully without crashing or corrupting data.
- SA-4.** All data entered in forms shall be validated before submission to prevent data corruption.
- SA-5.** The system shall provide appropriate warnings for potentially dangerous actions.
- SA-6.** Regular automated data backups shall be performed to prevent permanent data loss.
- SA-7.** The application shall maintain data consistency across all components.
- SA-8.** In case of server failure, the application shall display friendly error messages and suggest retry options.
- SA-9.** Critical operations (e.g., marks publication) shall require explicit user confirmation.
- SA-10.** The system shall log all errors for debugging and monitoring purposes.

19.5 Other Quality Attributes

19.5.1 Availability

- AV-1.** The system shall maintain 99.9% uptime during peak usage hours (8:00 AM - 8:00 PM).
- AV-2.** The system shall gracefully handle server failures with clear error messages.
- AV-3.** Scheduled maintenance shall be communicated to users at least 24 hours in advance.

19.5.2 Reliability

RE-1. The system shall handle network interruptions without crashing or losing data.

RE-2. The system shall maintain data consistency across all components.

RE-3. The mean time between failures (MTBF) shall be at least 720 hours.

19.5.3 Maintainability

MA-1. The code shall be modular to allow independent feature updates without affecting other components.

MA-2. Documentation shall be maintained for all components and APIs.

MA-3. Coding standards shall be followed for consistency across the codebase.

19.5.4 Scalability

SC-1. The backend architecture shall support horizontal scaling to handle increased user load.

SC-2. The database shall support efficient queries as data volume grows.

SC-3. The system shall handle peak loads without significant performance degradation.

19.5.5 Portability

PO-1. The application shall be compatible with Android 5.0+ (API 21) and iOS 12+.

PO-2. The application shall support multiple screen resolutions and sizes.

PO-3. The application shall support both portrait and landscape orientations.

19.5.6 Interoperability

IN-1. The system shall integrate with institutional email systems for authentication.

IN-2. The system shall use standard REST API protocols for external communication.

IN-3. The system shall support standard file formats (PDF, PPTX, DOCX).

20 Internationalization and Localization Requirements

The following requirements ensure that the application can be adapted for use in different regions and languages:

- IL-1.** The application shall support multiple languages (English and Bengali in v1.0).
- IL-2.** All user-facing text shall be stored in resource files for easy translation.
- IL-3.** Date and time formats shall follow local conventions (e.g., DD/MM/YYYY vs. MM/DD/YYYY).
- IL-4.** Unicode (UTF-8) shall be used for all text encoding.
- IL-5.** The interface shall support right-to-left (RTL) language rendering if needed.
- IL-6.** Currency symbols shall be localized if financial features are added.
- IL-7.** Time zones shall be handled appropriately for notification scheduling.
- IL-8.** Cultural considerations shall be respected in design elements and color choices.
- IL-9.** Localization shall not require code changes; all translations shall be externalized.
- IL-10.** The application shall support localized number and decimal formatting.
- IL-11.** Address formats shall be adaptable to local conventions.
- IL-12.** Localized keyboard layouts shall be supported for text input.

21 Other Requirements

21.1 Legal and Regulatory Compliance

- LR-1.** The system shall comply with institutional data protection policies and applicable privacy laws.
- LR-2.** User data shall be handled in accordance with applicable data protection regulations.
- LR-3.** Copyright shall be respected for all uploaded materials.
- LR-4.** The system shall display appropriate copyright and privacy notices.
- LR-5.** Terms of service and privacy policy shall be accessible within the application.
- LR-6.** User consent shall be obtained for data collection and processing.

21.2 Installation and Configuration

- IC-1.** The mobile application shall be distributed through official app stores (Google Play and Apple App Store).
- IC-2.** The backend shall be deployable on cloud infrastructure.
- IC-3.** Initial setup shall include creating administrative accounts.
- IC-4.** Configuration shall be manageable through environment variables or configuration files.
- IC-5.** The installation process shall be documented for system administrators.
- IC-6.** Updates shall be deployed without disrupting user access.

21.3 Logging and Monitoring

- LM-1.** The system shall log all critical actions (login, content upload, marks publication, approval actions).
- LM-2.** Error logs shall be maintained for debugging and troubleshooting purposes.
- LM-3.** System performance shall be monitored continuously.
- LM-4.** Security-related events shall be logged for audit purposes.
- LM-5.** Logs shall be retained for a minimum of 90 days.

LM-6. Monitoring alerts shall notify administrators of system issues.

21.4 Transition Requirements

TR-1. Data migration from existing systems shall be supported if needed.

TR-2. User training materials shall be provided for all user classes.

TR-3. A phased rollout strategy shall be implemented to ensure smooth adoption.

TR-4. Support shall be available during the transition period.

TR-5. Existing data shall be validated and cleaned before migration.

TR-6. Rollback procedures shall be documented and tested.

22 Specific Requirements

22.1 Functional Requirements

Requirement ID	Description	Priority
FR-01	The system shall allow users to log in using a valid institutional email address.	High
FR-02	The system shall verify the allowed email domain during login.	High
FR-03	The system shall authenticate users by verifying a securely hashed password.	High
FR-04	The system shall assign a role (Student, Teacher, CR, Admin, Office Staff) to each user upon successful login.	High
FR-05	The system shall display a role-specific home screen after login.	High
FR-06	The system shall allow the Admin to create and post official notices.	High
FR-07	The system shall allow the Class Representative (CR) to post notices specific to their batch.	High
FR-08	The system shall send a push notification to relevant users when a new notice is posted.	High
FR-09	The system shall allow a Teacher to create an assignment with a title, description, and deadline.	High
FR-10	The system shall send a push notification to enrolled students when an assignment is created.	High
FR-11	The system shall automatically send a 24-hour deadline reminder to students for each assignment.	High
FR-12	The system shall allow a Teacher to enter marks for their courses.	High
FR-13	The system shall allow a Teacher to save marks as a draft before publishing.	High
FR-14	The system shall allow a Teacher to publish marks, making them visible to students.	High
FR-15	The system shall allow a Student to view only their own published marks.	High
FR-16	The system shall allow a Teacher to upload lecture slides as PDF files, organized by course.	High

Requirement ID	Description	Priority
FR-17	The system shall allow a Teacher or Admin to upload reference books (PDFs) to a course library.	High
FR-18	The system shall allow an Admin to upload and update the class routine.	High
FR-19	The system shall send a push notification to all users when the class routine is updated.	High
FR-20	The system shall provide a course catalogue listing all courses with code, title, credits, semester, and teacher.	Medium
FR-21	The system shall allow a Teacher to set their weekly consultation schedule and update their next-day availability.	Medium
FR-22	The system shall allow a Student to submit a digital application.	High
FR-23	The system shall allow Office Staff to view submitted applications and forward them to the appropriate authority.	High
FR-24	The system shall allow authorized personnel to review, approve, or reject applications forwarded to them.	High
FR-25	The system shall send a push notification to the student when their application is approved or rejected.	High
FR-26	The system shall allow a Student to add their professional handles.	Low
FR-27	The system shall display student handles as clickable external links.	Low
FR-28	The system shall provide a dark mode and a light mode interface.	Medium
FR-29	The system shall persist the user's theme preference after the app is closed.	Medium
FR-30	The system shall allow a Teacher to create a short multiple-choice quiz for a course.	Medium
FR-31	The system shall allow a Student to attempt a published quiz and view their score.	Medium

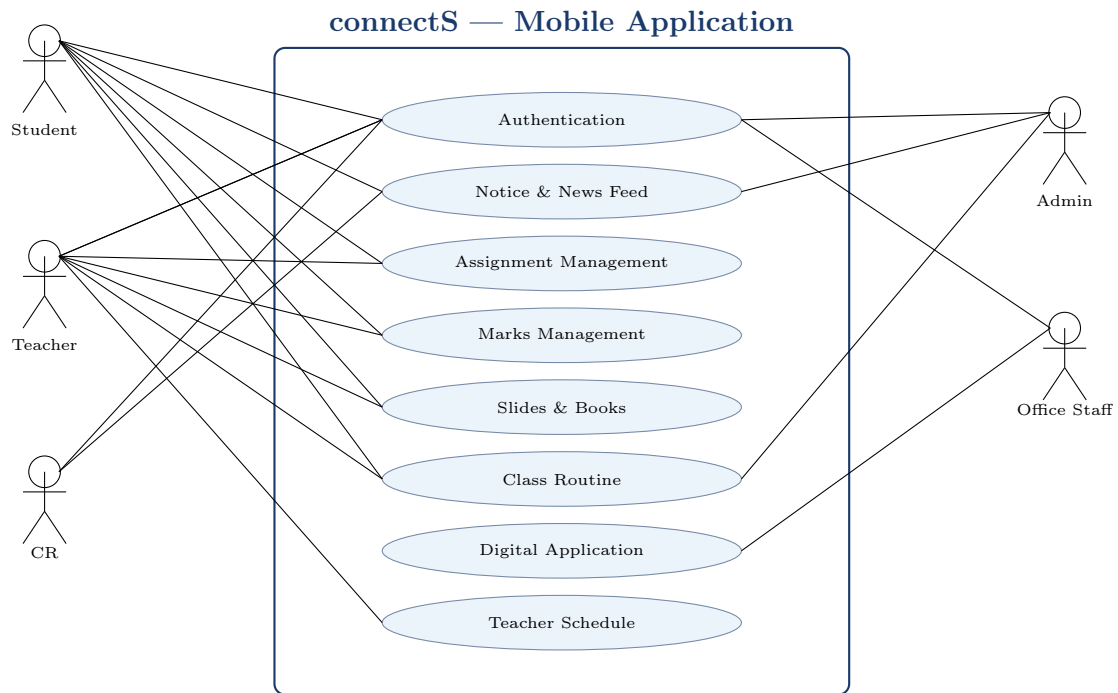
22.2 Non-Functional Requirements

Requirement ID	Description	Priority
NFR-01	The application's main screen shall load within 3 seconds on a stable 3G network.	High
NFR-02	The application shall be developed using the Flutter framework for cross-platform support (Android and iOS).	High
NFR-03	All passwords shall be stored using bcrypt hashing.	High
NFR-04	The system shall implement role-based access control to ensure users can only access data and functions for which they are authorized.	High
NFR-05	The user interface shall be simple and intuitive, requiring no training for a first-time user.	High
NFR-06	The system shall be available with a 99.9% uptime during peak usage hours.	Medium
NFR-07	All sensitive data transmitted between the app and the backend must be encrypted using TLS.	High
NFR-08	The system shall handle network errors gracefully, providing clear feedback to the user instead of crashing.	High
NFR-09	The system's data shall be backed up regularly to prevent data loss.	High
NFR-10	The mobile application shall be compatible with Android 5.0+ (API 21) and iOS 12+.	Medium
NFR-11	The application shall follow the Material Design guidelines for a consistent user experience.	Medium
NFR-12	All uploaded content shall be stored permanently and not be automatically deleted.	High
NFR-13	Push notifications should be delivered to the user within 60 seconds of the triggering event, under normal network conditions.	Medium
NFR-14	The system shall support dark and light themes without impacting performance or load time.	Low

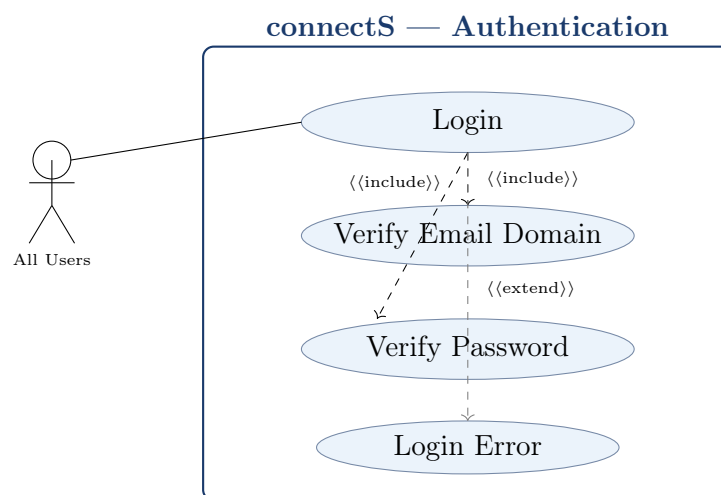
23 Scenario Based Modeling

23.1 Use Case Diagrams

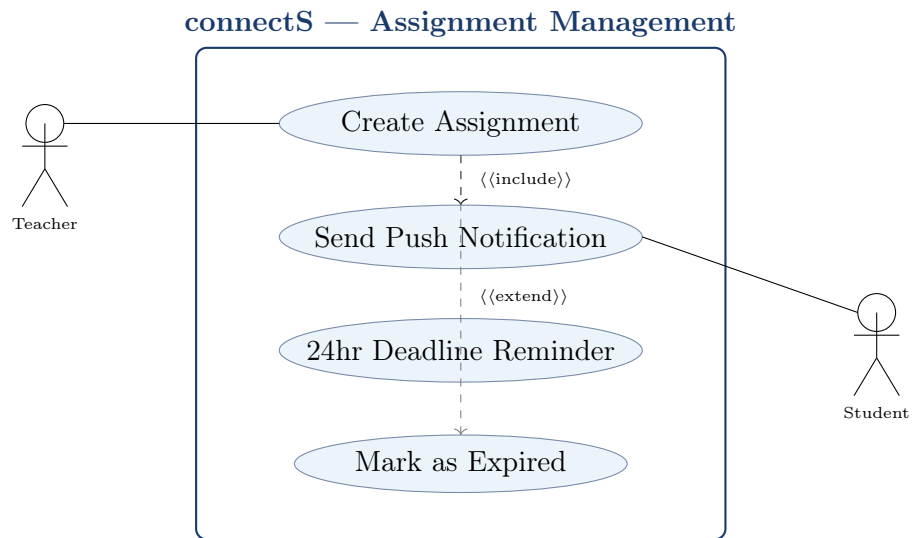
23.1.1 Level 0 — System Overview



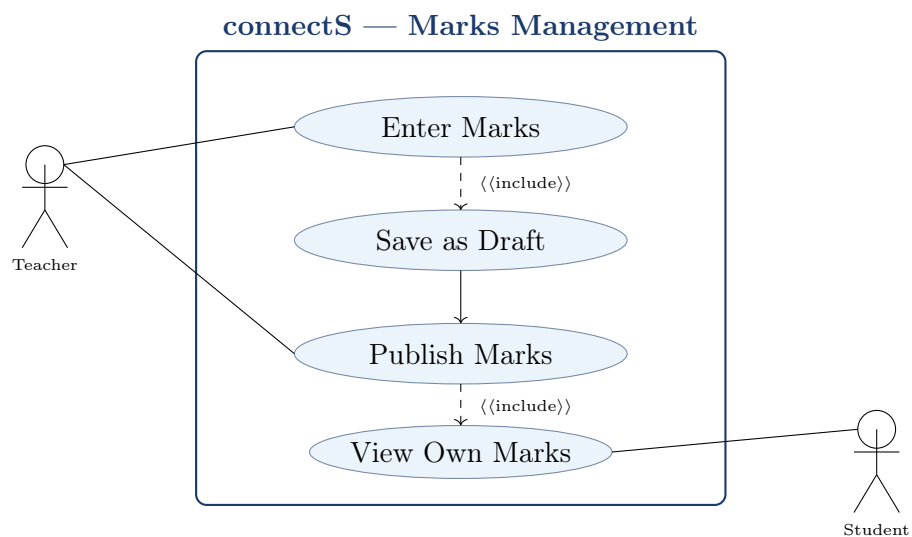
23.1.2 Level 1 — User Login and Authentication



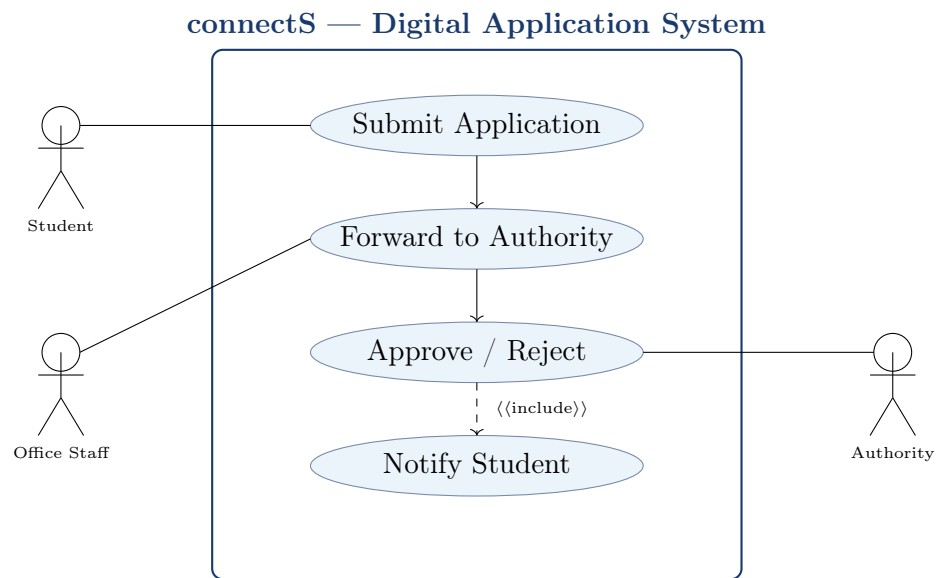
23.1.3 Level 2 — Assignment Deadline Management



23.1.4 Level 3 — Marks Management



23.1.5 Level 4 — Digital Application and Approval



24 Data and Class Based Modeling

24.1 Data Based Modeling

24.1.1 Identify Data Objects

- User • Notice • Assignment • Mark
- Slide • Book • ClassRoutine • Course
- Application • TeacherSchedule • Quiz • Notification

24.1.2 Final Data Objects

#	Data Object	Attributes
1	User	user_id, name, email, password_hash, role, batch, phone, profile_image, handles, created_at
2	Notice	notice_id, title, body, posted_by, target, batch_id, created_at
3	Assignment	assignment_id, title, description, course_id, teacher_id, deadline, is_expired, created_at
4	Mark	mark_id, student_id, course_id, teacher_id, assessment_type, score, is_published, published_at
5	Slide	slide_id, title, course_id, teacher_id, file_url, uploaded_at
6	Book	book_id, title, author, course_id, uploaded_by, file_url, uploaded_at
7	ClassRoutine	routine_id, semester, file_url, uploaded_by, uploaded_at, version
8	Course	course_id, code, title, credits, semester, prerequisites, teacher_id
9	Application	application_id, student_id, type, description, status, assigned_to, office_staff_id, comment, submitted_at, resolved_at
10	TeacherSchedule	schedule_id, teacher_id, weekly_slots, next_day_availability, updated_at
11	Quiz	quiz_id, title, course_id, teacher_id, questions, created_at
12	Notification	notif_id, user_id, title, body, type, is_read, created_at

24.2 Class Based Modeling

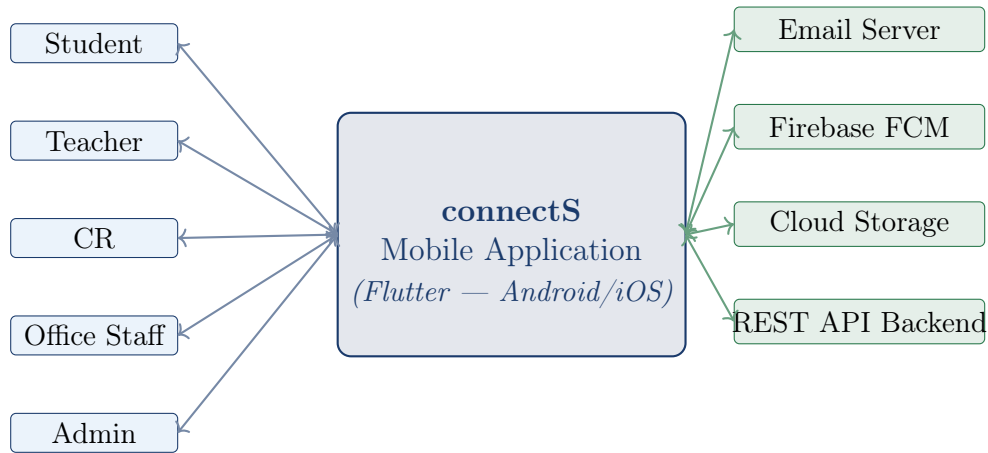
24.2.1 Final Classes

Class	Key Attributes	Key Methods
User	-user_id: int -name: string -email: string -password_hash: string -role: enum -batch: string	+login(): bool +logout(): void +updateProfile(): void +updateHandles(): void +getRole(): string
Notice	-notice_id: int -title: string -body: string -posted_by: int -target: enum -batch_id: string	+create(): void +publish(): void +delete(): void +getAll(): Notice[] +getByBatch(): Notice[]
Assignment	-assignment_id: int -title: string -description: string -course_id: int -deadline: datetime -is_expired: bool	+create(): void +update(): void +delete(): void +getActive(): Assignment[] +markExpired(): void
Mark	-mark_id: int -student_id: int -assessment_type: string -score: float -is_published: bool	+enter(): void +saveDraft(): void +publish(): void +getByStudent(): Mark[]
Slide	-slide_id: int -title: string -course_id: int -teacher_id: int -file_url: string	+upload(): void +delete(): void +getByCourse(): Slide[] +download(): void
Book	-book_id: int -title: string -author: string -course_id: int -file_url: string	+upload(): void +replace(): void +delete(): void +getByCourse(): Book[]

Class	Key Attributes	Key Methods
Course	-course_id: int -code: string -title: string -credits: int -semester: int -teacher_id: int	+add(): void +update(): void +archive(): void +search(): Course[] +filterBySemester(): Course[]
Application	-application_id: int -student_id: int -type: string -status: enum -assigned_to: int	+submit(): void +forward(): void +approve(): void +reject(): void +getStatus(): string
TeacherSchedule	-schedule_id: int -teacher_id: int -weekly_slots: JSON -next_day: JSON	+update(): void +updateNextDay(): void +getByTeacher(): Schedule
Notification	-notif_id: int -user_id: int -title: string -type: string -is_read: bool	+send(): void +markRead(): void +getUnread(): Notification[]
Quiz	-quiz_id: int -title: string -course_id: int -questions: JSON	+create(): void +publish(): void +attempt(): void +getResult(): float

25 Architectural Design

25.1 Representing the System in Context



25.2 Mapping Requirements to Software Architecture

Functional Requirement	Component	Technology / Approach
Institutional email login and role assignment	Auth Module	Email domain check, JWT session token, bcrypt password hashing
Push notifications for deadlines and notices	Notification Module	Firebase Cloud Messaging (FCM), background scheduler
Official notice board and CR news feed	Notice Module	REST API, role and batch-based access filter
Assignment creation and deadline tracking	Assignment Module	REST API, scheduled background job for reminders
Marks entry and publication	Marks Module	REST API, draft/publish state machine, student-ID filter
Lecture slide upload and in-app viewing	Slide Module	Cloud Storage, in-app PDF viewer
Books PDF library by course	Resource Module	Cloud Storage, lazy-loading PDF viewer
Class routine upload and update	Routine Module	Cloud Storage, Admin-only upload API, FCM on update

Functional Requirement	Component	Technology / Approach
Course catalogue with search and filter	Course Module	REST API, client-side keyword search and semester filter
Teacher schedule and consultation hours	Schedule Module	REST API, real-time polling for live updates
Digital application and approval workflow	Application Module	REST API, multi-step status machine, FCM at each stage
Student professional handles	Profile Module	REST API, dynamic URL builder, external browser link
Dark mode and light mode	Theme Module	Flutter ThemeData, Shared-Preferences for persistence
Mini quiz feature	Quiz Module	REST API, JSON question storage, auto-score calculation

26 Testing Plan

26.1 High-Level Description of Testing Goals

- **Bug Prevention:** Test each feature as soon as it is built to catch errors before they affect other parts of the system.
- **User Satisfaction:** Verify that the system behaves exactly as stakeholders described during elicitation interviews.
- **Security:** Confirm that role-based access control works correctly and no user can access another user's data.
- **Performance:** Verify that the app loads the main screen in under 3 seconds on a slow 3G network.
- **Reliability:** Test that the system stays available during peak usage hours and handles network errors gracefully.

26.2 Sample Test Cases

TC ID	Scenario	Steps	Expected Output	Actual Output	Result
TC-01	Login with valid institutional email	Enter valid email and correct password, tap Login	User reaches role-specific home screen	Same as expected	Pass
TC-02	Login with invalid email domain	Enter an email from a different domain, tap Login	Error: "Only allowed email domains are accepted"	Same as expected	Pass
TC-03	Student tries to access another student's marks	Log in as Student A, try to fetch Student B's marks via API	Access denied; only own marks returned	Same as expected	Pass

TC ID	Scenario	Steps	Expected Output	Actual Output	Result
TC-04	Teacher publishes marks	Enter marks, save as draft, tap Publish	Marks visible to students; push notification sent	Same as expected	Pass
TC-05	CR posts batch notice	Log in as CR, write notice, post	Notice visible only to own batch	Same as expected	Pass
TC-06	Student submits application	Fill application form, tap Submit	Administrative dashboard shows new entry	Same as expected	Pass
TC-07	Authority approves application	Open assigned application, tap Approve	Status changes to Approved; student receives notification	Same as expected	Pass
TC-08	Dark mode toggle and persistence	Open settings, enable dark mode, close and reopen app	Dark mode applied on reopen	Same as expected	Pass

Conclusion

Preparing this Software Requirements Specification has been a challenging and rewarding experience. The report covers the complete requirements and design modeling of **connectS**, a Flutter-based mobile application that addresses real and verified problems faced by academic institutions and departments.

Through stakeholder interviews with students, teachers, administrators, and staff, combined with surveys, the team gathered and confirmed 12 in-scope features, 31 functional requirements, and 14 non-functional requirements. The use case diagrams, data object models, class-based models, and architectural design presented in this report provide a clear and structured foundation for the development phase.

The unique findings from elicitation — such as the digital application and approval system identified from administrative staff interviews, and the dark/light mode requested independently by all stakeholder groups — show that the requirements gathering process was thorough and effective.

connectS is not only an academic exercise. It is a direct response to problems that students, teachers, and administrators face every day. With careful implementation following this specification, the application will significantly improve academic communication, resource access, and administrative efficiency for academic institutions.

References

- [1] Pressman, R. S., & Maxim, B. R. (2015). *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.
- [2] IEEE. (1998). *IEEE Recommended Practice for Software Requirements Specifications* (IEEE Std 830-1998). IEEE Computer Society.
- [3] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education Limited.
- [4] Flutter Development Team. (2026). *Flutter Documentation*. Retrieved from <https://flutter.dev/docs>
- [5] Firebase. (2026). *Firebase Cloud Messaging Documentation*. Retrieved from <https://firebase.google.com/docs/cloud-messaging>
- [6] Wiegers, K., & Beatty, J. (2013). *Software Requirements* (3rd ed.). Microsoft Press.

Appendix A: Glossary

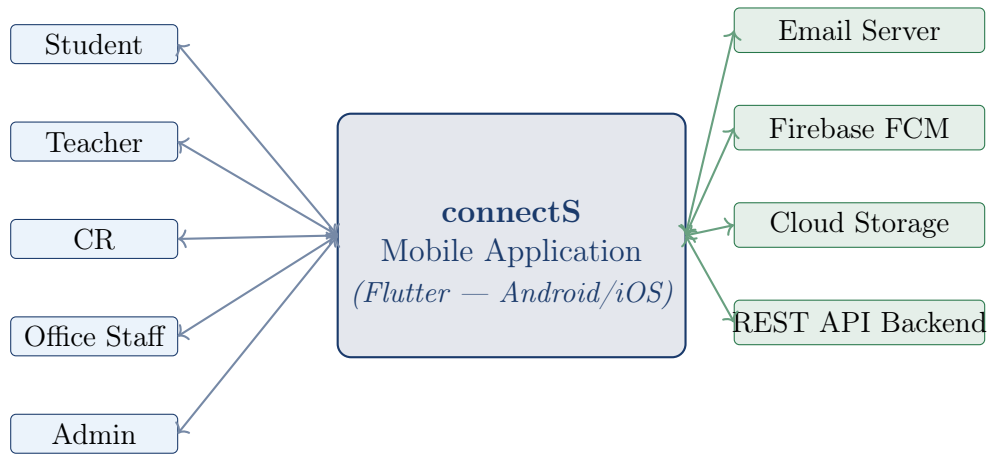
Term	Definition
Admin	User role responsible for system administration, user management, content moderation, and system configuration.
API	Application Programming Interface; defines how software components interact and exchange data.
Assessment	Evaluation of student performance (e.g., Class Test, Continuous Assessment, quizzes).
Batch	A group of students enrolled in the same academic program, year, and section.
bcrypt	A cryptographic hashing function used for securely storing passwords.
CA	Continuous Assessment; ongoing evaluation of student performance throughout the semester.
CR	Class Representative; a student elected to represent their batch and act as a liaison between students and faculty.
CT	Class Test; periodic tests administered during the semester to assess student learning.
Data Dictionary	A centralized repository of information about data elements and their relationships.
FCM	Firebase Cloud Messaging; a cross-platform messaging solution for delivering push notifications.
Flutter	An open-source UI software development toolkit by Google for building cross-platform applications.
FR	Functional Requirement; a specific behavior or function the system must provide.
HTTPS	Hypertext Transfer Protocol Secure; an extension of HTTP for secure communication over a network.
JSON	JavaScript Object Notation; a lightweight data interchange format.
JWT	JSON Web Token; an open standard for securely transmitting information between parties as a JSON object.
Lazy Loading	A design pattern where content is loaded only when needed to improve performance.
Localization	The process of adapting software for a specific region or language.

Term	Definition
Material Design	A design language developed by Google for creating consistent user interfaces.
MoSCoW	A prioritization method: Must have, Should have, Could have, Won't have.
NFR	Non-Functional Requirement; defines quality attributes and constraints of the system.
PDF	Portable Document Format; a file format used for presenting documents.
PUC	Product Use Case; describes a specific interaction between a user and the system.
QFD	Quality Function Deployment; a method for translating customer needs into technical requirements.
REST	Representational State Transfer; an architectural style for designing networked applications.
Role	A user classification that determines access permissions and available features.
SRS	Software Requirements Specification; this document.
TLS	Transport Layer Security; a cryptographic protocol for secure communication over a network.
UFTB	University of Frontier Technology, Bangladesh.
Unicode	A standard for consistent encoding, representation, and handling of text in most writing systems.

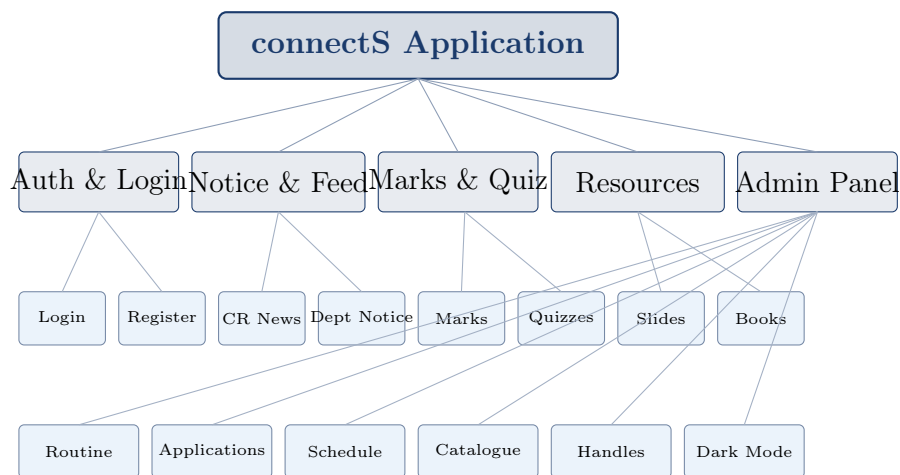
Appendix B: Analysis Models

This appendix contains analysis models that support the requirements specification.

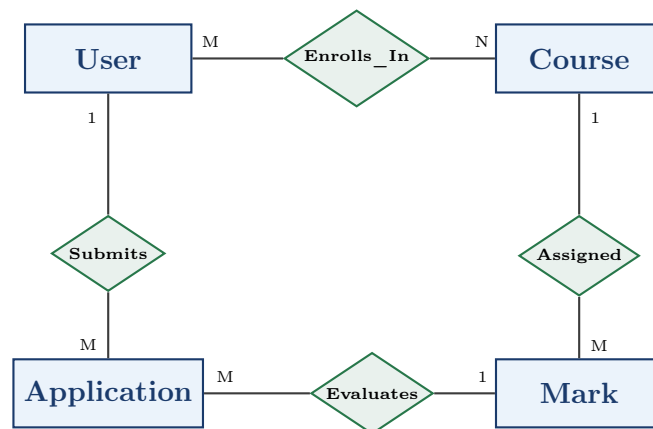
26.3 System Architecture in Context



26.4 Component Architecture



26.5 Entity-Relationship Diagram



26.6 Use Case Diagram — System Overview

